
AI Recruitment Assistant Project Documentation

Author: Vibha Balakrishna

Registration number:26BRS1058

VITemail:vibha.balakrishna2026@vitstudent.ac.in

1. Introduction

This project implements Part 1 (Extraction) of the AI-powered recruitment assistant assignment. The goal is to analyse conversational, unstructured text describing a person's experience or skills and extract structured information – skills, technologies, and languages – in JSON format. Per the assignment constraints, no LLM API is used; extraction relies entirely on rule-based NLP techniques.

Part 2 was optional and not implemented within the project timeline.

2. Problem Statement

Input: Conversational text describing experience (not a structured resume).

Example: *"I worked in the AI/ML Department and worked with CNN Models using Python"*

Output: A JSON object containing three categories – skill, technology, and language – extracted from the input text.

3. Approach and Design Decisions

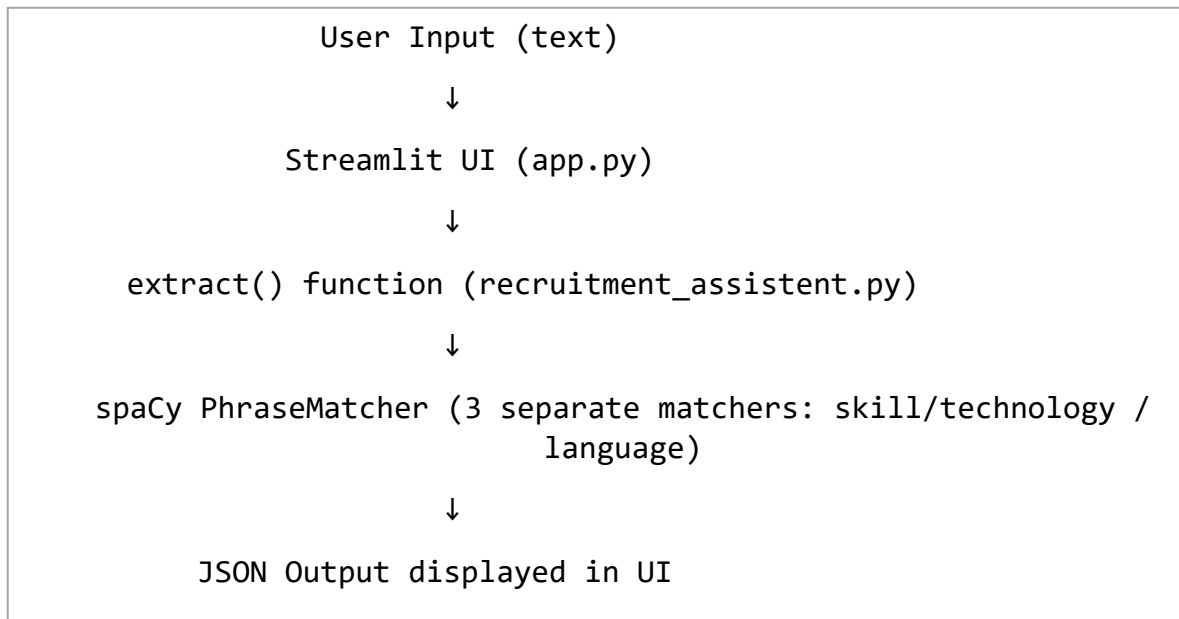
Why spaCy PhraseMatcher, not a trained NER model: Training a custom named-entity recognition model requires labelled training data and significant time, which wasn't feasible in the project timeframe. PhraseMatcher performs fast, accurate matching against a known vocabulary, which suits this problem well since skills/technologies/languages come from a fairly well-defined, enumerable set of terms.

Why no LLM API: Per assignment constraints. This meant extraction accuracy depends on vocabulary coverage rather than contextual/semantic understanding – a deliberate trade-off explained further in Limitations.

Vocabulary-driven matching: Three curated lists (SKILLS, TECHNOLOGIES, LANGUAGES) cover common terms across AI/ML, software development, cloud, and general professional skills. Case-insensitive matching (attr="LOWER") ensures robustness to input casing.

Chatbox interface: Built with Streamlit to satisfy the "chatbox" requirement – a live text input that returns extracted JSON immediately, rather than a static script.

4. System Architecture



Files:

recruitment_assistant.py – vocabulary lists + core extract() logic

app.py – Streamlit UI layer

requirements.txt – dependency list

5. Example Input/Output

Input: "I worked in the AI/ML Department and worked with CNN Models using Python"

Output:

json

```
{
  "skill": ["ai/ml"],
  "technology": ["cnn", "python"],
  "language": ["python"]
}
```

Input: "My name is vibha im currently working as an ai and robotics engineer with python and java"

Output:

```
json
{
  "skill": ["ai", "robotics"],
  "technology": ["python"],
  "language": ["java", "python"]
}
```

6. Tech Stack

Python 3.13

spaCy (en_core_web_sm)

Streamlit

7. How to Run

```
pip install -r requirements.txt
```

```
python -m spacy download en_core_web_sm
```

```
streamlit run app.py
```

8. Limitations

Relies on predefined vocabulary lists — terms not included won't be detected, unlike an LLM-based approach that could infer meaning from context.

No disambiguation between overlapping categories (e.g., "Python" appears in both technology and language lists by design, since it's valid in both contexts).

No handling of misspellings or unconventional phrasing beyond exact/case-insensitive phrase matches.

9. Future Work

Expand vocabulary lists further or source from a public taxonomy (e.g., ESCO, O*NET) for broader coverage.

Implement Part 2: resume skill extraction, job-role suggestion, and candidate-to-job-description matching using local embedding models (e.g., sentence-transformers) – no LLM API required.

10.CODE:

app.py

```
import streamlit as st
from recruitment_assistent import extract

st.title("AI Recruitment Assistant")
st.write("Describe your experience, skills, or background below:")

user_input = st.text_input("Your input:")

if user_input:
    result = extract(user_input)
    st.subheader("Extracted Information")
    st.json(result)
```

recruitment_assistent.py

```
import spacy
from spacy.matcher import PhraseMatcher

# ----- DATA -----
SKILLS = [
    "machine learning", "deep learning", "nlp", "natural language
processing",
```

```
"ai/ml", "ai", "ml", "data analysis", "data science", "computer vision",  
    "artificial intelligence", "neural networks", "backend development",  
    "frontend development", "web development", "cloud computing",  
    "devops", "database management", "api development", "testing",  
    "project management", "problem solving",  
    "communication", "robotics",  
    "data visualization", "big data", "data mining", "reinforcement learning", "iot", "edge computing",  
    "cybersecurity", "blockchain", "quantum computing", "engineering",  
    "software engineering", "mobile development", "game development", "leadership", "teamwork",  
    "critical thinking", "adaptability", "time management", "automation", "data engineering",  
    "data warehousing", "etl", "data modeling", "algorithms", "data structures",  
    "object-oriented programming", "functional programming", "api design", "microservices",  
    "containerization", "orchestration", "cloud architecture", "cloud security", "networking",  
    "virtualization", "serverless computing", "agile methodologies", "scrum", "kanban", "ui/ux design",  
    "user experience", "user interface design", "version control", "git", "github",  
    "continuous integration", "continuous deployment", "testing frameworks", "unit testing",  
    "integration testing", "performance optimization", "scalability"  
]
```

```
TECHNOLOGIES = [  
    "python", "cnn", "rnn", "lstm", "tensorflow", "pytorch", "keras",  
    "scikit-learn", "opencv", "react", "node.js", "django", "flask",  
    "docker", "kubernetes", "aws", "azure", "gcp", "git", "mysql",
```

```

    "postgresql", "mongodb", "spacy", "pandas", "numpy", "html",
    "css",

    "rest api", "graphql", "linux","pytorch lightning", "fastapi",
    "streamlit",

    "flask-restful", "celery","google cloud", "amazon web services",
    "microsoft azure",

    "heroku", "digital ocean","firebase", "oracle cloud", "ibm cloud",
    "cloudflare",

    "vercel", "netlify","apache spark", "hadoop", "kafka", "airflow",
    "jenkins", "ansible"
]

```

```

LANGUAGES = [

    "python", "java", "c++", "c", "javascript", "typescript", "sql",

    "r", "go", "rust", "swift", "kotlin", "php", "ruby",
    "c#", "matlab", "scala", "perl",

    "haskell", "lua", "dart", "elixir", "clojure","shell scripting",
    "bash", "powershell",

    "assembly", "fortran", "cobol","html", "css", "xml","html",
    "css3", "json", "yaml","markdown",

    "latex", "graphql","vhdl", "verilog", "f#","objective-c",
    "racket", "smalltalk","prolog",

    "lisp", "scheme","ada", "pascal", "delphi","julia", "nim",
    "crystal","groovy", "apex",

    "abap","sas", "stata", "spss","scratch", "logo", "basic"

]

```

```

# ----- EXTRACTOR -----

```

```

nlp = spacy.load("en_core_web_sm")

```

```

def make_matcher(terms):

```

```

    matcher = PhraseMatcher(nlp.vocab, attr="LOWER")

```

```

    patterns = [nlp.make_doc(t) for t in terms]

```

```
    matcher.add("TERMS", patterns)

    return matcher


skill_matcher = make_matcher(SKILLS)
tech_matcher = make_matcher(TECHNOLOGIES)
lang_matcher = make_matcher(LANGUAGES)


def get_matches(doc, matcher):
    spans = [doc[start:end].text for _, start, end in matcher(doc)]
    return sorted(set(s.lower() for s in spans))


def extract(text):
    doc = nlp(text)
    return {
        "skill": get_matches(doc, skill_matcher),
        "technology": get_matches(doc, tech_matcher),
        "language": get_matches(doc, lang_matcher)
    }


# ----- TEST -----
if __name__ == "__main__":
    test = input("Enter your experience/skills: ")
    print(extract(test))
```


RESULT:

1st input-output:

AI Recruitment Assistant

Describe your experience, skills, or background below:

Your input:

my name is vibha im currently working as an ai and robotics engineer with python and java

Extracted Information ↔

```
{
  "skill": [
    0 : "ai"
  ]
  "technology": [
    0 : "python"
  ]
  "language": [
    0 : "java"
    1 : "python"
  ]
}
```

2nd input-output:

Your input:

I've been doing data analysis and machine learning using pandas, numpy, and scikit-learn, mostly in R and Python

Extracted Information

```
{
  "skill": [
    0 : "data analysis"
    1 : "machine learning"
  ]
  "technology": [
    0 : "numpy"
    1 : "pandas"
    2 : "python"
    3 : "scikit-learn"
  ]
  "language": [
    0 : "python"
    1 : "r"
  ]
}
```

3rd input-output:

Your input:

I have strong problem solving and communication skills, along with experience in cybersecurity and networking using

Extracted Information

```
{
  "skill": [
    0 : "communication"
    1 : "cybersecurity"
    2 : "networking"
    3 : "problem solving"
  ]
  "technology": [
    0 : "linux"
  ]
  "language": []
}
```